```
# ------------------------------
# Step 1: Import required libraries
# ------------------------------

import numpy as np         # For numerical operations like arrays, max, argmax
import gym                 # To create the FrozenLake environment

# ------------------------------
# Step 2: Create the environment
# ------------------------------

# Create a 4x4 FrozenLake environment
# is_slippery=False makes it deterministic (no random slips)
env = gym.make("FrozenLake-v1", is_slippery=False)

# Print environment details
print("Number of states:", env.observation_space.n)  # Total possible states (16)
print("Number of actions:", env.action_space.n)      # Total possible actions (4)

# ------------------------------
# Step 3: Initialize policy & value function
# ------------------------------

# Policy: for each state, choose a random action initially
policy = np.random.choice(env.action_space.n, size=env.observation_space.n)

# Value function: start with zeros for all states
value_table = np.zeros(env.observation_space.n)

# Discount factor (gamma) to prioritize immediate vs. future rewards
gamma = 0.9

# Convergence threshold for policy evaluation
theta = 1e-10
```

```
# ------------------------------
# Step 4: Policy Evaluation
# ------------------------------

def policy_evaluation(policy, value_table, env, gamma=0.9, theta=1e-10):
  """
  Evaluate the value function under the current policy.
  Iteratively update state values until convergence.
  """
  while True:
    delta = 0  # Tracks maximum change in value during an iteration
    # Loop through all states
    for state in range(env.observation_space.n):
      v = 0
      action = policy[state]  # Action chosen by current policy
      # Look at possible transitions for this (state, action)
      for prob, next_state, reward, done in env.P[state][action]:
        # Bellman expectation equation
        v += prob * (reward + gamma * value_table[next_state])
      # Track max value change
      delta = max(delta, abs(value_table[state] - v))
      value_table[state] = v  # Update value of state
    # Stop if values converge (small change)
    if delta < theta:
      break
  return value_table


# ------------------------------
# Step 5: Policy Improvement
# ------------------------------
```

```
def policy_improvement(value_table, policy, env, gamma=0.9):
    """
    Improve the policy using the current value function.
    """
    policy_stable = True
    # Loop through all states
    for state in range(env.observation_space.n):
        old_action = policy[state]  # Current action
        action_values = np.zeros(env.action_space.n)  # Store values of all actions
        # Try all possible actions
        for action in range(env.action_space.n):
            for prob, next_state, reward, done in env.P[state][action]:
                action_values[action] += prob * (reward + gamma * value_table[next_state])
        # Choose action with highest value
        policy[state] = np.argmax(action_values)
        # Check if policy changed
        if old_action != policy[state]:
            policy_stable = False
    return policy, policy_stable


# ------------------------------
# Step 6: Policy Iteration
# ------------------------------

def policy_iteration(env, gamma=0.9, theta=1e-10):
    """
    Run policy iteration algorithm.
    Returns optimal policy and value function.
    """
    # Initialize policy and value table
    policy = np.random.choice(env.action_space.n, size=env.observation_space.n)
    value_table = np.zeros(env.observation_space.n)
```

```
    while True:
        # Step 1: Evaluate current policy
        value_table = policy_evaluation(policy, value_table, env, gamma, theta)

        # Step 2: Improve policy based on new values
        policy, policy_stable = policy_improvement(value_table, policy, env, gamma)

        # Stop if policy no longer changes
        if policy_stable:
            return policy, value_table


# ------------------------------
# Step 7: Run Policy Iteration
# ------------------------------

optimal_policy, optimal_value = policy_iteration(env, gamma, theta)

print("\nOptimal Policy (best action at each state):")
print(optimal_policy)

print("\nOptimal Value Function (long-term value of each state):")
print(optimal_value.reshape(4,4))  # Reshape into 4x4 grid for readability
```